

BAB II

LANDASAN TEORI

2.1 Konsep Dasar Sistem Informasi dan Situs Web

Sistem adalah kumpulan elemen yang saling berinteraksi untuk mencapai tujuan tertentu. **Sistem informasi** merupakan kombinasi terorganisasi antara manusia, perangkat lunak, perangkat keras, jaringan komunikasi, dan sumber data yang mengumpulkan, mengolah, menyimpan, serta mendistribusikan informasi dalam suatu organisasi (Sommerville, 2026). Sistem informasi berperan penting dalam pengambilan keputusan karena menyajikan data yang telah diolah menjadi informasi yang bermakna (Pressman & Maxim, 2023).

Situs web (*website*) adalah sekumpulan halaman yang saling terhubung dan dapat diakses melalui jaringan internet dengan menggunakan peramban (Abidin et al., 2024). Berdasarkan sifat penyajiannya, situs web dibedakan menjadi dua:

- **Situs web statis**, yaitu situs yang kontennya tetap dan tersimpan dalam berkas HTML. Perubahan konten mengharuskan pengeditan berkas secara langsung.
- **Situs web dinamis**, yaitu situs yang kontennya dibangkitkan secara otomatis dari basis data pada saat halaman diminta, sehingga perubahan dapat dilakukan melalui antarmuka pengelolaan tanpa menyentuh kode sumber (Kharismatunnisaa et al., 2023).

Sistem yang dibangun dalam kerja praktek ini termasuk kategori **situs web dinamis**, karena seluruh konten berita, artikel, agenda, galeri, dan struktur organisasi disimpan dalam basis data MySQL dan ditampilkan kembali melalui *query* yang dijalankan oleh *framework* Laravel.

2.2 Sistem Manajemen Konten (*Content Management System*)

Content Management System (CMS) adalah perangkat lunak yang memungkinkan pengguna untuk membuat, mengelola, dan memodifikasi konten digital pada sebuah situs web tanpa memerlukan pengetahuan teknis tentang pemrograman (Abidin et al., 2024). Karakteristik utama sebuah CMS adalah sebagai berikut:

- Pemisahan antara **konten** yang tersimpan di basis data dan **tampilan** yang diatur oleh *template*;
- Adanya **panel administrasi** yang dilindungi oleh autentikasi;
- Dukungan **manajemen pengguna dan hak akses**;
- Dukungan **pengunggahan berkas** media.

Pembangunan situs web profil organisasi dengan memanfaatkan *framework* Laravel telah terbukti efektif meningkatkan akses informasi dan partisipasi anggota, sebagaimana ditunjukkan oleh penelitian pada Ikatan Mahasiswa Arosbaya (Abidin et al., 2024) dan pada modul profil Dinas Cipta Karya, Tata Ruang dan Pertanahan (Kharismatunnisaa et al., 2023).

Sistem PKPT IPNU IPPNU UNSIQ dirancang sebagai CMS yang dibangun khusus (*custom CMS*), bukan menggunakan CMS siap pakai, karena struktur organisasi IPNU dan IPPNU memerlukan pengelompokan anggota berdasarkan tingkat (*level*) dan departemen yang tidak tersedia pada CMS umum.

2.3 Rekayasa Perangkat Lunak dan Model Pengembangan

Rekayasa perangkat lunak (*software engineering*) adalah disiplin ilmu yang menangani seluruh aspek produksi perangkat lunak, mulai dari tahap awal penentuan kebutuhan hingga pemeliharaan sistem setelah digunakan (Sommerville, 2026). Tahapan tersebut dirangkum dalam **SDLC** (*Software Development Life Cycle*) yang umumnya terdiri atas perencanaan, analisis kebutuhan, perancangan, implementasi, pengujian, dan pemeliharaan (Pressman & Maxim, 2023).

Model pengembangan yang digunakan pada kerja praktek ini adalah **model Waterfall**, yaitu model yang menyajikan alur proses secara berurutan dari satu tahap ke tahap berikutnya (Olindo & Syaripudin, 2022; Hafsari et al., 2024). Tahapan Waterfall yang diterapkan adalah sebagai berikut:

- **Analisis kebutuhan**, yaitu identifikasi kebutuhan fungsional berupa daftar modul dan hak akses, serta kebutuhan non-fungsional berupa kecepatan akses, keamanan, dan kemudahan penggunaan.
- **Perancangan**, yaitu pemodelan dengan UML, perancangan basis data, dan perancangan antarmuka.

- **Implementasi**, yaitu penulisan kode dengan Laravel 12 dan React 19.
- **Pengujian**, yaitu *black-box testing* dengan acuan karakteristik mutu ISO/IEC 25010:2023.
- **Pemeliharaan**, yaitu penyesuaian konten dan perbaikan yang bersifat berkesinambungan.

2.4 Pemodelan Sistem Berorientasi Objek dengan UML

2.4.1 Pengertian UML

Unified Modeling Language (UML) adalah bahasa pemodelan standar yang digunakan untuk memvisualisasikan, menspesifikasikan, membangun, dan mendokumentasikan artefak dari sistem berbasis perangkat lunak (Object Management Group [OMG], 2017). UML dikelola oleh *Object Management Group* dan versi yang berlaku saat ini adalah UML 2.5.1 (OMG, 2017).

UML mengelompokkan diagram ke dalam dua kategori besar, sebagaimana disajikan pada Tabel 2.1.

Tabel 2.1 Kategori Diagram UML

Kategori	Diagram	Fokus Pemodelan
Struktur (<i>structural</i>)	<i>Class diagram, Object diagram, Component diagram, Deployment diagram</i>	Hal-hal yang bersifat statis, yaitu elemen penyusun sistem dan hubungan antar elemennya
Perilaku (<i>behavioral</i>)	<i>Use case diagram, Activity diagram, Sequence diagram, State machine diagram</i>	Hal-hal yang bersifat dinamis, yaitu proses, alur, dan interaksi yang terjadi di dalam sistem

Sumber: Object Management Group (2017)

Pada perancangan sistem ini digunakan empat diagram yang paling lazim dipakai dalam pemodelan sistem informasi, yaitu *use case diagram*, *activity diagram*, *sequence diagram*, dan *class diagram* (Hafsari et al., 2024).

2.4.2 *Use Case Diagram*

Use case diagram memodelkan **apa** yang dilakukan sistem dari sudut pandang pengguna, tanpa menjelaskan **bagaimana** sistem melakukannya. Diagram ini

digunakan pada tahap awal untuk mengidentifikasi fungsi-fungsi sistem dan pihak yang berhak menggunakannya. Komponen dan notasi *use case diagram* disajikan pada Tabel 2.2.

Tabel 2.2 Notasi *Use Case Diagram*

Elemen	Notasi	Keterangan
<i>Actor</i>	Simbol orang (stikman)	Peran yang berinteraksi dengan sistem, dapat berupa manusia atau sistem lain
<i>Use case</i>	Elips	Fungsi atau layanan yang disediakan sistem, ditulis dengan kata kerja
<i>System boundary</i>	Persegi	Batas yang memisahkan sistem dengan lingkungan luar
<i>Association</i>	Garis lurus tanpa panah	Hubungan komunikasi antara aktor dan <i>use case</i>
<i>Include</i> <<include>>	Garis putus-putus berpanah, dari <i>use case</i> dasar menuju <i>use case</i> yang disertakan	Perilaku yang selalu dilakukan apabila <i>use case</i> dasar dijalankan
<i>Extend</i> <<extend>>	Garis putus-putus berpanah, dari <i>use case</i> tambahan menuju <i>use case</i> dasar	Perilaku opsional atau bersyarat yang menambah <i>use case</i> dasar
<i>Generalization</i>	Garis lurus dengan panah segitiga berongga, dari yang spesifik ke yang umum	Pewarisan sifat antar aktor atau antar <i>use case</i>

Sumber: Object Management Group (2017)

Hal yang paling sering keliru pada *use case diagram* adalah arah panah <<include>> dan <<extend>>. Keduanya menggunakan garis putus-putus berpanah, tetapi arahnya berlawanan:

- <<include>> diartikan sebagai perilaku wajib, sehingga panah **menuju** *use case* yang disertakan. Contohnya, *Kelola Berita* <<include>> *Autentikasi* berarti setiap kali *Kelola Berita* dijalankan, *Autentikasi* pasti dijalankan.

- <<extend>> diartikan sebagai perilaku opsional, sehingga panah **berasal dari use case** tambahan **menuju use case** dasar. Contohnya, *Muat Ulang Konten Realtime* <<extend>> *Melihat Beranda* berarti pembaruan hanya terjadi apabila ada konten baru, tidak selalu.

Cara mudah mengingatnya adalah: **include** berarti wajib sehingga panah keluar dari induk, sedangkan **extend** berarti opsional sehingga panah masuk ke induk.

2.4.3 *Activity Diagram*

Activity diagram menggambarkan alur kerja (*workflow*) dari sebuah proses. Diagram ini menyerupai *flowchart*, tetapi memiliki kemampuan untuk memodelkan proses yang berjalan secara paralel. Notasi utama *activity diagram* disajikan pada Tabel 2.3.

Tabel 2.3 Notasi *Activity Diagram*

Elemen	Notasi	Fungsi
<i>Initial node</i>	Lingkaran penuh	Titik awal, hanya boleh ada satu
<i>Activity</i>	Persegi panjang bersudut tumpul	Satu langkah atau aksi dalam proses
<i>Decision</i>	Belah ketupat	Percabangan berdasarkan kondisi
<i>Fork</i>	Garis tebal horizontal	Memecah alur menjadi beberapa alur paralel
<i>Join</i>	Garis tebal horizontal	Menggabungkan kembali alur yang paralel
<i>Final node</i>	Lingkaran berisi titik penuh	Titik akhir proses

Sumber: Object Management Group (2017)

Kesalahan yang umum terjadi pada *activity diagram* adalah melupakan *initial node* atau *final node*, serta menggunakan simbol percabangan tanpa keterangan kondisi pada garis percabangannya.

2.4.4 *Sequence Diagram*

Sequence diagram menggambarkan interaksi antar objek yang diurutkan berdasarkan waktu. Diagram ini menjawab pertanyaan mengenai objek apa yang mengirim pesan kepada objek apa, dan pada urutan ke berapa. Komponen

sequence diagram disajikan pada Tabel 2.4.

Tabel 2.4 Notasi *Sequence Diagram*

Elemen	Notasi	Keterangan
<i>Lifeline</i>	Garis vertikal putus-putus di bawah objek	Menyatakan keberadaan objek sepanjang waktu
<i>Activation bar</i>	Persegi panjang sempit di atas <i>lifeline</i>	Selang waktu objek sedang menjalankan operasi
<i>Synchronous message</i>	Garis lurus dengan ujung panah penuh	Pengirim menunggu balasan
<i>Asynchronous message</i>	Garis lurus dengan ujung panah terbuka	Pengirim tidak menunggu balasan
<i>Return message</i>	Garis putus-putus dengan ujung panah terbuka	Nilai kembalian dari suatu operasi
<i>Self-message</i>	Panah melengkung kembali ke <i>lifeline</i> yang sama	Objek memanggil operasinya sendiri

Sumber: Object Management Group (2017)

Kesalahan yang umum terjadi pada *sequence diagram* adalah menggambar pesan balasan (*return*) dengan garis lurus, serta tidak menyertakan *activation bar* sehingga tidak tampak kapan sebuah objek sedang aktif.

2.4.5 *Class Diagram*

Class diagram menggambarkan struktur statis sistem, yaitu kelas-kelas penyusun sistem beserta atribut, metode, dan hubungan antar kelasnya. Setiap kelas digambarkan sebagai persegi panjang yang dibagi menjadi tiga bagian, yaitu nama kelas pada bagian atas, atribut pada bagian tengah, serta metode atau operasi pada bagian bawah. Notasi visibilitas yang digunakan disajikan pada Tabel 2.5.

Tabel 2.5 Notasi Visibilitas pada *Class Diagram*

Simbol	Arti	Keterangan
+	<i>Public</i>	Dapat diakses dari kelas lain
-	<i>Private</i>	Hanya dapat diakses di dalam kelas itu sendiri
#	<i>Protected</i>	Dapat diakses oleh kelas itu sendiri dan kelas turunannya

Sumber: Object Management Group (2017)

Notasi hubungan antar kelas beserta maknanya disajikan pada Tabel 2.6.

Tabel 2.6 Notasi Hubungan Antar Kelas

Hubungan	Notasi	Makna
<i>Association</i>	Garis lurus	Hubungan struktural antar kelas
<i>Aggregation</i>	Garis dengan belah ketupat berongga di ujung kelas induk	Hubungan "memiliki" yang longgar; bagian dapat berdiri sendiri
<i>Composition</i>	Garis dengan belah ketupat penuh di ujung kelas induk	Hubungan "memiliki" yang kuat; bagian tidak dapat berdiri sendiri
<i>Generalization</i>	Garis dengan panah segitiga berongga menuju kelas induk	Hubungan pewarisan (<i>parent-child</i>)
<i>Dependency</i>	Garis putus-putus berpanah	Satu kelas bergantung pada kelas lain

Sumber: Object Management Group (2017)

Setiap hubungan dilengkapi **multiplisitas** yang menyatakan jumlah objek yang terlibat, yaitu 1 untuk tepat satu, 0..1 untuk nol atau satu, 1.. untuk satu atau lebih, dan 0.. untuk nol atau lebih.

2.4.6 Perbedaan *Class Diagram* dan *Entity Relationship Diagram*

Bagian ini perlu ditegaskan karena *class diagram* dan ERD sering tertukar. Keduanya sama-sama berbentuk kotak dan saling berhubungan, tetapi objek yang dimodelkan berbeda. Perbedaannya disajikan pada Tabel 2.7.

Tabel 2.7 Perbedaan *Class Diagram* dan ERD

Aspek	<i>Class Diagram</i>	<i>Entity Relationship Diagram (ERD)</i>
Objek yang dimodelkan	Kelas perangkat lunak	Entitas data atau tabel
Isi kotak	Nama, atribut, dan metode	Nama entitas dan atribut saja
Baris ketiga pada kotak	Ada, berisi metode	Tidak ada
Relasi	Asosiasi, agregasi, komposisi, generalisasi	Relasi dengan kardinalitas (1:1, 1:N, N:M)
Kunci	Tidak mengenal <i>primary key</i> dan <i>foreign key</i>	Mengenal PK dan FK secara eksplisit
Termasuk diagram UML	Ya, diagram struktur	Bukan, berasal dari pemodelan basis data relasional

Sumber: Object Management Group (2017)

Implikasi praktisnya bagi laporan ini adalah sebagai berikut. Apabila yang digambar berupa kotak berisi daftar kolom tabel tanpa baris metode, maka diagram tersebut adalah **ERD**, bukan *class diagram*, dan harus diletakkan pada sub-bab perancangan basis data. *Class diagram* yang benar memuat metode, seperti yang ditunjukkan pada contoh berikut.

```
+-----+
|           User           |
+-----+
| - id: int                |
| - name: string           |
| - email: string          |
| - password: string       |
| - role: enum             |
+-----+
| + member(): HasOne       |
| + posts(): HasMany      |
| + articles(): HasMany   |
| + isAdmin(): bool       |
| + isKetua(): bool       |
| + isDepartemen(): bool  |
+-----+
```

2.4.7 Kesalahan Umum dalam Pemodelan UML

Kesalahan yang paling sering ditemukan pada pemodelan UML beserta perbaikannya disajikan pada Tabel 2.8.

Tabel 2.8 Kesalahan Umum Pemodelan UML dan Perbaikannya

No	Kesalahan	Perbaikan
1	Arah panah <<include>> dan <<extend>> terbalik	<code>include</code> berarti wajib sehingga panah keluar dari induk; <code>extend</code> berarti opsional sehingga panah masuk ke induk
2	Menyebut ERD sebagai <i>class diagram</i>	ERD untuk basis data; <i>class diagram</i> memuat metode dan bukan merupakan diagram UML
3	<i>Use case</i> terlalu rinci, yaitu setiap operasi CRUD dijadikan <i>use case</i> terpisah	Gabungkan menjadi satu <i>use case</i> "Kelola Berita"; perincian CRUD ditulis pada skenario <i>use case</i>

No	Kesalahan	Perbaikan
4	Tidak ada <i>system boundary</i>	Tambahkan persegi pembatas sistem
5	Aktor diberi nama orang	Aktor adalah peran, misalnya "Admin", bukan nama orang
6	<i>Activity diagram</i> tanpa <i>initial node</i> atau <i>final node</i>	Setiap alur harus memiliki satu titik awal dan minimal satu titik akhir
7	<i>Sequence diagram</i> tanpa <i>activation bar</i>	Sertakan agar tampak kapan objek aktif
8	<i>Return message</i> digambar dengan garis lurus	<i>Return</i> harus bergaris putus-putus dengan panah terbuka
9	Multiplisitas tidak ditulis pada <i>class diagram</i>	Tulis pada kedua ujung setiap relasi
10	Menyalin definisi tanpa menyebut sumber	Setiap definisi wajib disertai kutipan

Sumber: Object Management Group (2017); Hafsari et al. (2024)

2.5 Arsitektur Perangkat Lunak: MVC dan Inertia.js

Pola MVC (*Model–View–Controller*) memisahkan aplikasi menjadi tiga komponen. Ketiga komponen tersebut beserta implementasinya pada sistem ini disajikan pada Tabel 2.9.

Tabel 2.9 Komponen Pola MVC dan Implementasinya

Komponen	Tanggung Jawab	Implementasi pada Sistem Ini
<i>Model</i>	Mengelola data, aturan bisnis, dan akses basis data	Kelas Eloquent pada direktori <code>app/Models</code>
<i>View</i>	Menyajikan data kepada pengguna	Komponen React pada direktori <code>resources/js/Pages</code>
<i>Controller</i>	Menerima permintaan, memanggil <i>model</i> , meneruskan ke <i>view</i>	Kelas pada direktori <code>app/Http/Controllers</code>

Sumber: Sommerville (2026)

Pemisahan ini memudahkan pemeliharaan karena perubahan tampilan tidak mengharuskan perubahan pada logika akses data, dan sebaliknya (Sommerville,

2026).

Pada arsitektur lama, sisi *backend* dan *frontend* dipisahkan menjadi dua aplikasi yang berkomunikasi melalui REST API, sehingga pengembang harus menulis, mendokumentasikan, dan merawat dua lapisan sekaligus. Pendekatan **Inertia.js** menawarkan cara yang berbeda, yaitu *controller* pada sisi server tidak mengembalikan JSON, melainkan langsung mengembalikan nama komponen *frontend* berikut data yang diperlukannya. Hasilnya, aplikasi tetap berupa satu kesatuan (*monolith*) dengan satu basis kode, satu *route*, dan satu sesi autentikasi, tetapi pengguna tetap merasakan pengalaman aplikasi satu halaman (*Single Page Application*) karena navigasi tidak memuat ulang halaman secara penuh (Stauffer, 2023; Rizzi et al., 2025).

Single Page Application (SPA) adalah konsep arsitektur aplikasi web yang membuat cara kerja situs web menyerupai aplikasi desktop, di mana peramban tidak melakukan pemuatan ulang secara terus-menerus pada keseluruhan halaman saat bernavigasi, melainkan hanya memuat ulang komponen yang berubah (Rizzi et al., 2025).

2.6 Teknologi Pengembangan

2.6.1 PHP

PHP (*PHP: Hypertext Preprocessor*) adalah bahasa skrip *server-side* yang banyak digunakan untuk pengembangan aplikasi web (Kharismatunnisaa et al., 2023). Sistem ini mensyaratkan PHP 8.2 atau yang lebih baru.

2.6.2 Laravel 12

Laravel adalah *framework* PHP yang menerapkan pola MVC dan menyediakan berbagai fasilitas siap pakai, antara lain *routing*, ORM, autentikasi, sistem migrasi basis data, dan *middleware* (Kharismatunnisaa et al., 2023; Stauffer, 2023). Sistem ini menggunakan **Laravel 12**, versi utama yang dirilis pada 24 Februari 2025. Komponen Laravel yang dimanfaatkan disajikan pada Tabel 2.10.

Tabel 2.10 Komponen Laravel dan Penggunaannya pada Sistem

Komponen	Fungsi	Penggunaan pada Sistem Ini
Eloquent ORM	Pemetaan objek ke tabel basis data	Sembilan model, yaitu User, Post, Article, Agenda, Gallery, Member, Quote, Statistic, PageSetting, dan SiteSetting
<i>Migration</i>	Pengelolaan skema basis data secara berversi	Dua puluh lima berkas migrasi yang merekam seluruh perubahan skema
<i>Middleware</i>	Penyaringan permintaan sebelum mencapai <i>controller</i>	auth, verified, dan role
<i>Observer</i> dan <i>Event</i>	Penanganan kejadian pada model	ContentObserver memicu <i>event</i> ContentUpdated
<i>Broadcasting</i>	Pengiriman <i>event</i> ke klien secara <i>realtime</i>	Kanal public-content
Breeze	<i>Scaffolding</i> autentikasi	Login, registrasi, verifikasi email, dan atur ulang kata sandi

Sumber: Stauffer (2023); Anugra et al. (2025)

Eloquent ORM bekerja dengan prinsip bahwa setiap tabel direpresentasikan oleh satu kelas model, sehingga operasi basis data dilakukan dengan sintaks objek dan tidak memerlukan penulisan SQL secara manual (Saputra & Zakaria, 2023).

2.6.3 MySQL

MySQL adalah sistem manajemen basis data relasional (*RDBMS*) yang menyimpan data dalam bentuk tabel-tabel yang saling berhubungan (Anugra et al., 2025). Basis data sistem ini bernama pkpt dan ditata dengan relasi *primary key* dan *foreign key* serta indeks pada kolom yang sering dijadikan saringan, yaitu *event_date*, *is_active*, dan *published_at*.

2.6.4 React 19

React adalah pustaka JavaScript *open-source* yang dikembangkan untuk membangun antarmuka pengguna berbasis komponen yang dapat digunakan kembali (*reusable components*) (Rizzi et al., 2025). React memanfaatkan *Virtual DOM* yang hanya memperbarui komponen yang berubah, sehingga

sistem yang dikembangkan menjadi lebih cepat dan efisien (Rizzi et al., 2025). Sistem ini menggunakan **React 19** yang dipadukan dengan Vite sebagai *build tool*. Pustaka pendamping yang digunakan disajikan pada Tabel 2.11.

Tabel 2.11 Pustaka Pendamping React dan Fungsinya

Pustaka	Fungsi
Tailwind CSS	Kerangka CSS berbasis utilitas untuk perancangan antarmuka
Framer Motion	Pustaka animasi antarmuka
Lucide React	Pustaka ikon SVG
SweetAlert2	Dialog notifikasi dan konfirmasi
Laravel Echo	Penerima <i>broadcast realtime</i> di peramban

Sumber: Rizzi et al. (2025)

2.6.5 Cloudinary

Cloudinary adalah layanan pengelolaan media berbasis awan (*cloud*) yang menangani pengunggahan, penyimpanan, optimasi, dan pengiriman gambar (Anugra et al., 2025). Penggunaan layanan *cloud* berbeda dengan penyimpanan berkas lokal, karena berkas tidak tersimpan di *server* aplikasi melainkan pada jaringan *server* terdistribusi, sehingga waktu muat gambar lebih singkat dan beban *server* aplikasi berkurang. Pada sistem ini, seluruh berkas gambar, yaitu logo situs, gambar berita, foto galeri, dan foto anggota, dikelola melalui Cloudinary (Anugra et al., 2025).

2.6.6 Pusher

Pusher adalah layanan *WebSocket* terkelola yang memungkinkan *server* mengirimkan pembaruan ke peramban secara langsung tanpa adanya permintaan dari klien, sehingga perubahan hak akses maupun konten dapat diperbarui secara langsung tanpa memuat ulang halaman (Ramadhani et al., 2026). Mekanisme yang diterapkan pada sistem ini adalah sebagai berikut:

- Pengelola menambah atau mengubah konten melalui panel admin;
- `ContentObserver` menangkap kejadian `created`, `updated`, atau `deleted` pada model;
- `Event ContentUpdated` disiarkan ke kanal `public-content`;

- Peramban pengunjung yang sedang membuka halaman menerima *event* tersebut dan memuat ulang data secara otomatis.

2.7 Keamanan: Autentikasi, Otorisasi, dan RBAC

Keamanan perangkat lunak mencakup upaya melindungi sistem dan data dari akses yang tidak sah. Terdapat dua konsep yang perlu dibedakan, yaitu **autentikasi** yang memastikan siapa pengguna yang mengakses sistem, dan **otorisasi** yang menentukan apa yang boleh dilakukan pengguna setelah terautentikasi.

RBAC (*Role-Based Access Control*) adalah model pengendalian akses yang menetapkan hak berdasarkan peran pengguna dalam sistem, bukan pada masing-masing individu (Khairi & Alda, 2024; Ramadhani et al., 2026). Dengan RBAC, penambahan pengguna baru hanya memerlukan penetapan peran, bukan penyusunan ulang daftar hak akses. Penerapan RBAC terbukti mampu membatasi akses pengguna sesuai perannya dan mencegah akses yang tidak berizin (Ramadhani et al., 2026).

Sistem ini menerapkan RBAC dengan tiga peran yang dipisahkan pada tingkat *middleware*, sebagaimana disajikan pada Tabel 2.12.

Tabel 2.12 Peran Pengguna dan Modul yang Dapat Diakses

Peran	Modul yang Dapat Diakses
admin	Seluruh modul, termasuk manajemen pengguna dan pengaturan situs
ketua	Agenda, galeri, anggota, dan statistik
departemen	Berita, artikel, dan kutipan

Sumber: Khairi dan Alda (2024)

2.8 Pengujian Perangkat Lunak

Pengujian perangkat lunak adalah proses menjalankan sistem dengan tujuan menemukan kesalahan, kekurangan, atau ketidaksesuaian antara hasil yang diperoleh dengan hasil yang diharapkan (Sommerville, 2026).

Black-box testing adalah teknik pengujian yang mengamati fungsionalitas sistem hanya dari sisi masukan dan keluaran tanpa memeriksa struktur kode di dalamnya (Ayu et al., 2024; Ningrum et al., 2024). Teknik ini sesuai digunakan

untuk memastikan bahwa setiap fitur berjalan sesuai dengan kebutuhan pengguna (Hasibuan et al., 2025).

Sebagai acuan penyusunan instrumen pengujian, digunakan **ISO/IEC 25010:2023**, yaitu standar internasional edisi kedua yang mendefinisikan model mutu produk perangkat lunak (ISO, 2023). Standar ini memuat sembilan karakteristik mutu beserta sub-karakteristiknya, sebagaimana disajikan pada Tabel 2.13.

Tabel 2.13 Karakteristik Mutu ISO/IEC 25010:2023

Karakteristik	Sub-karakteristik
<i>Functional suitability</i>	<i>Functional completeness, functional correctness, functional appropriateness</i>
<i>Performance efficiency</i>	<i>Time behaviour, resource utilisation, capacity</i>
<i>Compatibility</i>	<i>Co-existence, interoperability</i>
<i>Usability</i>	<i>Appropriateness recognisability, learnability, operability, user error protection, user interface aesthetics, accessibility</i>
<i>Reliability</i>	<i>Maturity, availability, fault tolerance, recoverability</i>
<i>Security</i>	<i>Confidentiality, integrity, non-repudiation, accountability, authenticity</i>
<i>Maintainability</i>	<i>Modularity, reusability, analysability, modifiability, testability</i>
<i>Portability</i>	<i>Adaptability, installability, replaceability</i>

Sumber: International Organization for Standardization (2023)

Pada pengujian sistem ini, karakteristik yang diukur dibatasi pada *functional suitability*, *usability*, dan *performance efficiency*, sebagaimana lazim dilakukan pada pengujian situs web dengan pendekatan *black-box testing* (Ayu et al., 2024; Supriyanto et al., 2026).

DAFTAR PUSTAKA

Buku

Pressman, R. S., & Maxim, B. R. (2023). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sommerville, I. (2026). *Software engineering, global edition* (10th ed.). Pearson Education.

Stauffer, M. (2023). *Laravel: Up & running: A framework for building modern PHP apps* (3rd ed.). O'Reilly Media.

Majalah, Jurnal Ilmiah, dan Prosiding

Abidin, Z., Kurniawan, J., Suntoro, D. A. P., & Yuliasuti, G. E. (2024). Rancang bangun website profil Ikatan Mahasiswa Arosbaya menggunakan Laravel dan Bootstrap 5 dengan metode waterfall. *Prosiding Seminar Implementasi Teknologi Informasi dan Komunikasi (SEMTIK)*, 3(2).

Anugra, A. D., Nasir, M., Herdiansyah, M. I., & Effendy, I. (2025). Integrasi Midtrans payment gateway pada website e-commerce Hafiz Cell. *BETRIK: Besemah Teknologi Informasi dan Komputer*, 16(3). <https://doi.org/10.36050/1tgahm98>

Ayu, K. G., Sari, D. W., Zein, A., & Fadila, R. S. (2024). Pengujian black-box dan analisa kualitas ISO 25010 pada aplikasi HESTI. *CESS (Journal of Computer Engineering, System and Science)*, 9(2), 710–721. <https://doi.org/10.24114/cess.v9i2.61621>

Hafsari, R., Arribe, E., Andria, M. L., & Miransya, V. (2024). Perancangan sistem informasi penjualan menggunakan metode waterfall (studi kasus PT. Riau Pos Intermedia). *PROSISKO: Jurnal Pengembangan Riset dan Observasi Sistem Komputer*, 11(1). <https://doi.org/10.30656/prosisko.v11i1.7794>

Hasibuan, D. B., Pinem, A. A. A., & Ramadhani, F. (2025). Pengujian kualitas website sistem pembelajaran digital (SIPADI POLMED) menggunakan black box testing dan standar ISO/IEC 25010. *Jurnal Cakrawala Akademika*, 2(1), 1027–1033.

Khairi, A. S., & Alda, M. (2024). Implementasi role based access control dalam pengelolaan hak akses koperasi berbasis mobile. *Jurnal Teknik Informatika Unika St. Thomas*, 9, 85–95.

Kharismatunnisaa, F., Saputra, Y., Bahri, S., & Siskandar, R. (2023). Penerapan framework Laravel pada modul profil untuk website Dinas Cipta Karya,

Tata Ruang dan Pertanahan (DCKTRP). *JSI: Jurnal Sains Indonesia*, 4(3), 249–263. <https://doi.org/10.59897/jsi.v4i3.175>

Ningrum, M., Bisri, M., & Hulmi, Z. A. (2024). Pengukuran kualitas sistem informasi profil kampus berbasis web menggunakan model ISO 25010 dan metode black-box testing. *Jurnal Ilmu Komputer, Sistem Informasi, Teknik Informatika*, 3(1), 67–80.

Olindo, V., & Syaripudin, A. (2022). Perancangan sistem informasi absensi pegawai berbasis web dengan metode waterfall. *OKTAL: Jurnal Ilmu Komputer dan Science*, 1(1), 17–26.

Ramadhani, S., Armiady, D., & Fajri, R. (2026). Penerapan role-based access control (RBAC) untuk mengelola hak akses pada sistem informasi sekolah taman kanak-kanak berbasis multiuser. *Jurnal Ilmu Komputer Aceh*, 3(1), 129–141.

Rizzi, T. A., Abdillah, R. A., Yancandra, Y. E., Wijaya, M., & Voutama, A. (2025). Implementasi React Vite dalam pengembangan antarmuka sistem pemesanan tiket pesawat dengan metode Scrum. *PROCESSOR*, 20(1).

Saputra, S., & Zakaria, H. (2023). Implementasi object relational mapper pada aplikasi daily report berbasis web menggunakan Laravel Eloquent. *LOGIC: Jurnal Ilmu Komputer dan Pendidikan*, 2(1), 22–33.

Supriyanto, A., Aji, B., Haidar, A., Oktavian, I., & Sahono, M. N. (2026). Pengujian karakteristik usability pada website katalog barang menggunakan pendekatan black-box testing berdasarkan standar ISO/IEC 25010. *Modem*, 4(3). <https://doi.org/10.62951/modem.v4i3.968>

Elektronik

International Organization for Standardization. (2023). *ISO/IEC 25010:2023 systems and software engineering—systems and software Quality Requirements and Evaluation (SQuaRE)—product quality model* (2nd ed.). <https://www.iso.org/standard/78176.html>

Object Management Group. (2017). *OMG unified modeling language (OMG UML) version 2.5.1* (formal/2017-12-05). <https://www.omg.org/spec/UML/2.5.1/PDF>